DevDocs — Error / Loading / Empty States

1. Overview

DevDocs should provide clear, minimal UI states for situations where:

- A project has no files.
- Files exist but are not ready for chat.
- Files fail to process.
- Chat generation fails.
- Retrieval or internal processing fails.
- The application is starting.
- The assistant is processing a request.
- The user stops an in-progress generation.

The UI should remain minimal and avoid exposing unnecessary internal implementation details such as BM25, vector search, reranking, embeddings, or LLM provider errors.

---

2. Empty State — New Project With Zero Files

When a newly created project contains no files:

UI

Display a clear empty state:

> **No files have been added yet. Upload your project folder to get started.**

Provide:

Upload Project

Behavior

- This is an empty state, not an error.
- The user can upload the project folder.
- Chat remains unavailable until at least one file becomes `Ready`.

---

3. Files Exist but None Are Ready

When files exist in the project but none have reached `Ready` status:

Chat UI

Keep the chat interface visible but disabled.

Display:

> **No files are ready yet.**

Do not display detailed processing counts in the chat area.

Processing status is communicated through:

- Document Explorer file statuses.
- The global ingestion/processing indicator when applicable.

If files are still processing, the existing ingestion indicator communicates that separately.

---

4. All Files Failed

When all project files have failed processing:

Chat UI

Keep the chat interface visible but disabled.

Display:

> **No files are ready. Some files failed to process. Retry the failed files to start chatting.**

Detailed failure information is handled inside Document Explorer.

Retry

Each failed file has a **Retry** action in Document Explorer.

Retrying changes the file status:

```
Failed
  ↓
Processing
  ↓
Ready
```

or:

```
Failed
  ↓
Processing
  ↓
Failed
```

A failure in one file must not fail the entire project.

---

5. LLM / API Generation Failure

If answer generation fails after the configured automatic retries have been exhausted:

Display:

> **Couldn't generate a response. Please try again.**

Provide:

Retry

Behavior

- The original user message remains saved.
- No failed assistant message is saved.
- Any partial assistant response is not persisted.
- Retry uses the same original user message.
- Retry performs a **fresh RAG retrieval + fresh LLM generation**.
- Retry does not create a duplicate user message.

Technical details such as:

- HTTP 500
- provider errors
- timeout details
- API-specific error messages

are not shown in the normal user interface.

---

6. RAG / Internal Pipeline Failure

If an internal processing stage fails, such as:

- BM25 retrieval
- Vector retrieval
- Embedding generation
- RRF

- Reranking
- Vector database
- Other RAG pipeline components

the user should not be told which subsystem failed.

Use the same generic error state:

> **Couldn't generate a response. Please try again.**

Retry

Behavior

Retry:

```
Same user message
      ↓
Fresh RAG retrieval
      ↓
Fresh context construction
      ↓
Fresh LLM generation
```

The user should not need to understand the underlying failure.

---

7. Loading State — Before First Token

After the user sends a question, DevDocs performs retrieval, context construction, and begins LLM generation.

Before the first LLM token arrives, display a short rotating status message.

Examples:

- `Thinking...`
- `Levitating...`
- `Connecting the dots...`
- `Digging in...`
- `Figuring it out...`
- `Reading between the lines...`
- `Piecing it together...`
- `On it...`
- `Almost there...`

Important

These messages are **purely cosmetic**.

They do not represent actual backend stages.

Do not display technical statuses such as:

- `Searching files...`
- `Running BM25...`
- `Running vector search...`
- `Reranking...`
- `Building context...`
- `Calling LLM...`

This keeps the UI simple and prevents the interface from exposing implementation details.

Transition to Streaming

Once the first LLM token arrives:

```
Loading phrase
     ↓
Disappears
     ↓
Assistant response begins streaming
```

---

8. Interrupted Generation — Stop

While the assistant is streaming a response, the user can click **Stop**.

Behavior

When Stop is clicked:

1. Cancel the active generation.
2. Keep the already-generated partial response visible temporarily.
3. Mark the response as stopped.
4. Display a small `Stopped` indicator.
5. Display a `Retry` action.
6. Do not save the partial assistant response.
7. Keep the original user message saved.

Example:

```
The authentication flow starts by checking
the JWT token and then...
```

```
[Stopped] [Retry]
```

The partial response exists only as temporary UI state.

---

9. Leaving an Interrupted Chat

If the user leaves the chat while an interrupted/partial assistant response is present, the partial response is not persisted.

When the user returns to the conversation:

```
User:
How does authentication work?

[Stopped] [Retry] [Cancel]
```

The previously displayed partial assistant response is gone.

Important

The original user message remains in the conversation because user messages are persisted regardless of generation success.

The interrupted assistant response is never stored as a normal assistant message.

---

10. Stopped Query — Retry

If the user clicks **Retry** on a stopped query:

```
Original user message
       ↓
Fresh RAG retrieval
       ↓
Fresh context construction
       ↓
Fresh LLM generation
       ↓
Stream response
```

The system does not resume the previous partial generation.

The previous partial assistant output is discarded.

The user message is not duplicated.

---

11. Stopped Query — Cancel

A stopped user query can also be canceled.

When the user clicks **Cancel**:

```
User:
How does authentication work?

[Stopped] [Retry] [Cancel]
```

becomes:

```
User:
How does authentication work?
```

Behavior

- The stopped/retry state is removed.
- The user message itself is not deleted.
- The query is not retried.
- No assistant response is generated.
- The user can continue with a new query.

`Cancel` means:

> Do not retry this interrupted query.

It does **not** mean:

> Delete the user message.

---

12. Sending a New Query After an Interruption

The user does not need to explicitly cancel a stopped query before asking something new.

Example:

```
Previous query
[Stopped] [Retry] [Cancel]
```

```
User enters:

Where is the database configuration?

Send
```

The new query becomes the next query to process.

The previous stopped state is no longer an active pending-generation state.

The new query follows the normal processing pipeline:

```
New user query
      ↓
Save user message
      ↓
Fresh project-scoped RAG
      ↓
Build context
      ↓
LLM generation
      ↓
Stream response
      ↓
Save completed assistant response
```

The previous interrupted assistant generation is never resumed.

---

13. Core Rule for Interrupted Generations

> **A stopped generation is abandoned. The next submitted query always starts a fresh generation.**

There are two ways to resolve an interrupted query:

Option 1 — Cancel

```
Stopped query
      ↓
Cancel
      ↓
No retry
      ↓
User can ask a new question
```

Option 2 — Send a new query

```
Stopped query
      ↓
User enters new query
      ↓
Send
      ↓
Fresh generation for the new query
```

---

14. App Startup State

When DevDocs is opened, display a short splash screen.

Sequence

```
Application starts
       ↓
DevDocs logo displayed
       ↓
Approximately 2 seconds
       ↓
Logo fades out
       ↓
Application fades in
```

UI

- Center the DevDocs logo.
- No loading text.
- No spinner.
- Use a smooth fade transition.
- Target approximately 3 seconds for the splash experience.

The splash screen is purely visual and should not expose backend initialization details.

---

15. State Summary

Situation	UI State	User Action
New project, zero files	Empty state	Upload Project
Files exist, none Ready	Chat disabled	Wait for files to become Ready

Situation	UI State	User Action
All files failed	Chat disabled + failure message	Retry failed files
LLM/API failure	Generic error	Retry
RAG/internal failure	Generic error	Retry
Waiting for first token	Rotating playful status	Wait
Response streaming	Normal streaming response	Stop
Generation stopped	Partial response + Stopped + Retry	Retry / Cancel / ask new question
Return to chat after interruption	User message + Stopped + Retry + Cancel	Retry / Cancel / ask new question
App startup	Logo splash	Wait for application

16. UX Principles

Minimal UI

Do not expose unnecessary technical details to the user.

No internal pipeline leakage

Users should not need to know whether a failure occurred in:

- BM25
- Vector search
- RRF
- Reranking
- Embeddings
- Vector database
- LLM provider

Preserve conversation history

User messages remain persisted even when generation fails or is stopped.

Do not persist incomplete assistant responses

Only successfully completed assistant responses become normal conversation messages.

Fresh generation

Every retry or new query starts a fresh retrieval and generation process.

Clear recovery actions

Whenever possible, provide a simple action such as:

- **Retry**
- **Cancel**
- **Upload Project**

---

17. V1 Scope

Included:

- Project empty states
- File readiness states
- Failed-file recovery
- Generic generation errors
- Generic internal processing errors
- Automatic retries
- Rotating loading messages
- Streaming interruption handling
- Retry after interruption
- Cancel after interruption
- New-query handling after interruption
- Startup splash screen

Not included:

- Detailed technical error logs in the user UI
- Exposing individual RAG pipeline stages
- Persistent partial assistant responses
- Automatic continuation of stopped generations
- Resuming interrupted LLM generations
- Advanced crash-recovery UI